RepoAI

High-Level Design
.NET backend architecture

An open-source web application that transforms source repositories into animated architecture explanations and supports detailed, code-grounded conversation.

Document	Baseline
Version	1.1 - Swagger, Google sign-in and LTX revision
Updated	25 September 2026
Backend	C# / .NET 10 LTS / ASP.NET Core
Deployment	Docker images distributed through Docker Hub; Docker Compose for self-hosting
Scope	System boundaries, components, flows, data ownership, scaling and operational requirements

Architecture direction

Use a modular .NET backend with independent background workers. Analyze each repository snapshot once, reuse the resulting knowledge for chat and video, and deliver useful results progressively.

Technology boundary

RepoAI application services and business logic remain C#/.NET. LTX video generation runs as an isolated external model dependency; its official inference stack uses Python/PyTorch [12]. The .NET workers orchestrate this runtime through an adapter. Browser graphics and FFmpeg compose accurate overlays and final media. No Node.js application backend is required.

This is an HLD proposal, not an implementation or a performance benchmark. Detailed schemas, class interfaces and endpoint specifications belong in the subsequent LLD.

01 / Product scope and design goals

Primary user journey

A user imports a repository, sees analysis progress, explores the architecture, generates a narrated video, and asks detailed follow-up questions. Chat becomes available when sufficient repository evidence is indexed; it does not wait for the video to finish.

Capability	Expected behavior
Repository understanding	Identify components, entry points, dependencies, services and supported execution relationships. Pin all findings to a repository commit.
Architecture video	Use LTX models for generated animated clips, combined with readable code, verified diagrams, narration and evidence-linked overlays.
Detailed conversational chat	Explain code and architecture, answer follow-ups, trace relationships and discuss improvements with source citations and relevant video timestamps.
Open deployment	Publish application source and Docker images; support configurable open-model inference and persistent local storage.

Non-functional priorities

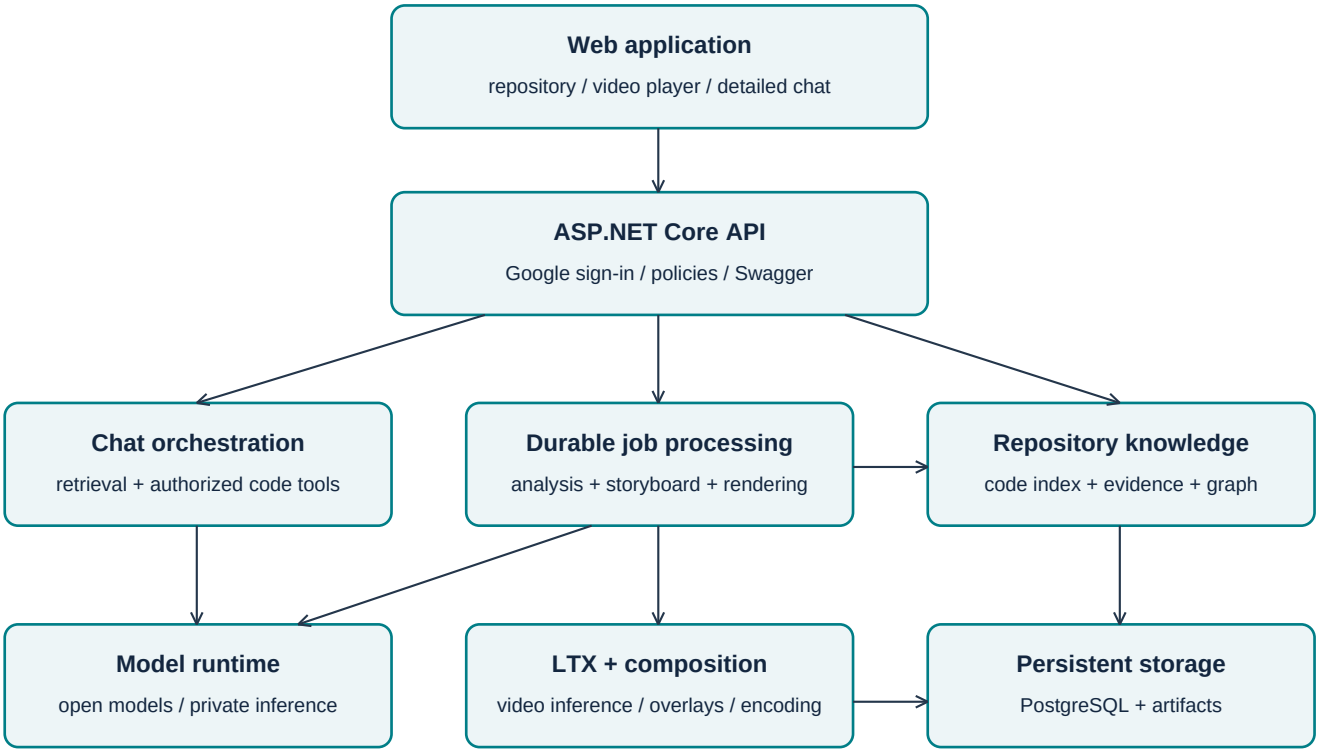
Responsiveness: stream answers, show progressive analysis, and keep long work off request threads. **Correctness:** attach evidence to repository claims and distinguish inferred behavior. **Recoverability:** resume failed stages without restarting the entire project. **Scalability:** scale CPU, model inference and rendering independently.

Boundaries and assumptions

Repository analysis cannot prove every runtime behavior, especially reflection, generated code and external service interactions. Report analysis coverage and unsupported languages. Begin with deep C# support and a language-adapter boundary for expansion; launch language breadth remains a product decision.

The initial video experience focuses on accurate explanatory 3D graphics. Film-quality photorealistic animation requires additional assets, rendering investment and art direction. The chosen model, GPU, concurrency level and voice model must be validated before performance commitments.

02 / System architecture



Arrows show primary interactions; chat and workers share repository evidence and model access.

Interactive path

The browser communicates with ASP.NET Core over HTTPS. The API handles identity, project access and commands. Chat orchestration retrieves relevant repository evidence, invokes a privately hosted model, and streams a detailed answer with verified source references.

Background path

The API persists long-running jobs and returns immediately. .NET workers perform repository analysis, planning and video orchestration. A separate LTX inference runtime generates clips. The .NET composition worker adds verified diagrams, source overlays, narration and captions before encoding the final video.

Shared foundation

Chat and video consume the same commit-pinned knowledge model. PostgreSQL stores application state, searchable evidence and job progress. Artifact storage holds source snapshots, scene specifications, audio and completed videos. Model and media runtimes remain private dependencies.

03 / Major components and technology choices

Component	Responsibility and proposed technology
Web application	Project workspace, source explorer, video player, scene preview and detailed chat. Frontend framework is independent of the .NET backend.
Backend API	ASP.NET Core 10; Google sign-in, RepoAI access policies, project commands and streaming. Built-in OpenAPI with Swagger UI for manual API testing.
Repository analysis	C# Worker Service with Roslyn for deep C# analysis, configuration parsers and future language adapters. Imported code is not executed.
Knowledge and search	PostgreSQL through EF Core/Npgsql; symbol, full-text and optional vector search; relationships stored as graph-shaped records.
Chat and model gateway	C# orchestration through Microsoft.Extensions.AI abstractions; bounded code-reading tools, conversation context and provider adapters.
Model runtime	Configurable private inference endpoint. Native llama.cpp is an initial option; final runtime/model selection follows capability and hardware tests.
Video planning	C# workers turn evidence into a structured narrative and validated scene specifications.
Video and composition	LTX generates clips through a separate model runtime. C# workers use Playwright .NET and FFmpeg for verified overlays, captions, narration alignment and final assembly.
Job processing	Durable PostgreSQL-backed jobs, stage dependencies, retries, leases and cancellation; independently configurable .NET workers.
Artifact storage	Filesystem volumes for one host; S3-compatible object storage for multi-host deployment.

Use .NET 10 LTS as the application baseline [1]. Keep the model adapter replaceable through Microsoft.Extensions.AI [2]. Start with a modular backend and explicit worker boundaries; extract more services only when workload or team ownership warrants it.

04 / End-to-end data flows

A. Repository import and progressive readiness

Import request → immutable snapshot → parsing → searchable evidence → architecture relationships → enrichment. Resolve the selected branch or tag to an exact commit. Fetch within size/time limits, parse supported files, publish a basic index, and then enrich with embeddings and summaries. The UI reports coverage and readiness at each stage.

B. Video generation

Repository evidence → storyboard and narration → LTX clip generation → verified overlays → final assembly. The planner supplies scene prompts and optional reference images to the LTX adapter. The worker validates generated media, aligns clips with the narration, and composes source-grounded diagrams and code. Independent scenes run within GPU limits; previews appear before final export.

LTX is the selected generative video engine. Generated visuals are illustrative, not evidence of the repository architecture. Exact labels, code and service connections are composed from verified data. The final video includes captions, a scene manifest and source mappings. Invalid clips receive bounded retries or a clearly reported scene failure.

C. Detailed chat

User question + conversation + optional video position → retrieval → bounded code tools → model response → streamed answer. Retrieve symbols, source spans, architecture relationships and relevant narration. Allow follow-up file reads or relationship tracing where needed. Provide explanations at the requested depth, retain conversational context, and attach file/line references and scene timestamps.

General programming questions can receive general explanations; repository-specific claims require evidence. An explanation of why an author made a decision is labeled as an inference unless documented in the repository.

D. Repository update

New commit → changed files and dependencies → selective reanalysis → new snapshot. Reuse unchanged artifacts where their inputs still match. Invalidate affected relationships, summaries and scenes. Existing videos and conversations remain tied to their original snapshot, so their citations do not silently change.

05 / Data, reliability and security

Data ownership

Data group	Purpose
Workspace and project	Google issuer/subject mapping to local user; membership, repository connection and RepoAI access policy.
Snapshot and evidence	Commit, files, symbols, dependencies, source spans and analysis coverage.
Conversation and video	Messages, citations, storyboards, scenes, timing and generation provenance.
Jobs and artifacts	Pipeline state, attempts, ownership, progress, immutable outputs and retention.

Reliable processing

Persist jobs together with their initiating state change. Workers claim bounded leases and heartbeat while running. Retry transient failures with backoff; stop on invalid input. Use idempotent stages and fenced publication so duplicate or stale workers cannot overwrite successful results. Persist cancellation and recover unfinished work after a restart.

Render scenes separately and publish the complete video only after required outputs pass validation. Preserve partial chat answers as incomplete if inference fails. Stream events can reconnect; the persisted final message or job state remains authoritative.

Cache and consistency

Cache by content hash, snapshot, model/prompt revision and rendering configuration. Scope private caches to the workspace. Immutable evidence keeps chat citations and video references consistent. When dependencies change, invalidate affected semantic analysis even if an individual file is unchanged.

Security boundaries

Authorize every project read, tool call, stream and artifact download. Restrict repository fetch destinations and reject path traversal. Treat code and README content as untrusted input, including instructions aimed at the model. Do not run imported builds or scripts by default. Rendering accepts approved templates and assets, not model-generated executable code.

Run containers with limited privileges and resources; keep credentials in secrets, redact logs, and isolate model endpoints. Deletion cancels work and removes dependent data and artifacts under a documented retention policy. Backups have a separate retention and restore policy.

06 / Docker deployment and scaling

Deployment unit	Role
repoai-api	ASP.NET Core API, chat orchestration and event streaming; can serve prebuilt frontend files.
repoai-worker	C# analysis and planning workers; queue selection controls workload ownership.
repoai-renderer	C# video orchestration/composition worker with pinned browser/media dependencies.
ltx-inference	Private LTX GPU runtime using its upstream Python/PyTorch stack. Separate image and model volume; reached through the .NET adapter.
model runtime	Private native inference service with mounted model weights and assigned compute.
PostgreSQL	Persistent application data, indexes and durable jobs.
Artifact volume / object store	Repository snapshots, audio, previews and video outputs.

Self-hosted baseline

Distribute versioned RepoAI images through Docker Hub and provide Docker Compose to assemble them. Keep model weights, secrets and user data out of image layers. Start with one Linux host, persistent volumes and configurable external model access. Document a tested GPU profile; Docker Compose GPU access depends on host setup [5].

Scale by bottleneck

Add analysis workers for CPU-heavy indexing, render workers for video backlog, and model capacity for inference demand. Scale stateless API replicas after shared session keys, admission control and streaming behavior are ready. Move artifacts to shared object storage before distributing workers across hosts.

Reserve inference capacity for chat and schedule LTX separately. A shared GPU may not fit both models simultaneously; serialize workloads or unload models when necessary. Dedicated chat and LTX GPUs are preferred for predictable latency. Measure model residency, LTX resolution/frame count and composition load before raising concurrency.

Operations

Expose liveness/readiness and separate model status. Collect traces, queue age, first-token latency, generation throughput, renderer utilization and failure rates. Stop claiming jobs during shutdown and allow leases to recover interrupted work. Use controlled database migrations, pinned release images, backups and periodic restore rehearsals.

07 / Performance goals and delivery plan

These are proposed targets for a warm deployment under a measured concurrency limit. They are not verified results; model, hardware, repository size and output length must accompany benchmark reports.

Metric	Initial design target
Ordinary API requests	p95 below 300 ms, excluding media transfer and external identity operations.
Indexed evidence retrieval	p95 below 500 ms on a defined reference repository set.
Simple chat first token	p95 below 3 seconds including admission and retrieval at supported load. Deep tool-assisted questions are measured separately.
Progress visibility	Updates visible within 2 seconds of a persisted stage change.
Import and video completion	Set targets after benchmarking repository sizes, scene complexity, profiles and hardware.

Implementation sequence

1. **Foundation:** .NET hosts, Google sign-in, access policies, Swagger UI, PostgreSQL and Docker.
2. **Repository intelligence:** immutable imports, parsing, search and architecture overview.
3. **Detailed chat:** model integration, bounded tools, streaming and citations.
4. **Video:** LTX adapter/runtime, storyboard, clip validation, verified overlays, narration, captions and export.
5. **Release:** recovery, security tests, benchmarks, license review and Docker Hub distribution.

Acceptance criteria

Evaluate analysis against known repositories; check factual answers and citation support; test chat while videos render; kill workers to verify recovery; test cancellation and reconnect; verify workspace isolation and malicious repository handling. Measure visual clarity and narration quality with representative users.

Decisions for the next design stage

Choose reference hardware, concurrency, coding/voice models, the exact LTX checkpoint and its license, and the first visual templates. Validate Google callback configuration and pinned Swagger/OpenAPI package compatibility. LTX memory and generation latency remain benchmark-dependent.

08 / Swagger API testing on .NET 10

Compatibility confirmed

Swagger UI is compatible with ASP.NET Core on .NET 10. Microsoft documents using the Swashbuckle.AspNetCore.SwaggerUI package to interact with documents generated by Microsoft.AspNetCore.OpenApi [9]. The removal of Swagger from default templates does not remove compatibility.

Selected integration

Use ASP.NET Core built-in OpenAPI generation as the single specification source and Swagger UI as the interactive test interface. Publish the document at /openapi/v1.json and the interface at /swagger in development. Pin .NET 10-compatible package versions and validate the generated OpenAPI version against the bundled UI. Do not enable two competing schema generators.

Testing workflow

Developers sign into RepoAI with Google in the same browser, then open the same-origin Swagger UI. Try it out requests use the existing RepoAI session. Configure the UI to attach the application anti-forgery header on state-changing calls. Google tokens are not pasted into Swagger as RepoAI API bearer tokens.

Document request/response models, authentication requirements, job identifiers, pagination and failure responses. Test repository import, job status, conversation creation and video commands through Swagger. Long operations return a job reference; the UI checks job status rather than holding a request open until video generation finishes.

Scope of validation

Swagger supports interactive manual API testing. Automated .NET integration tests still verify access policies, contract behavior, retries and regressions. Test SSE reconnect and media playback through dedicated clients or the application because the Swagger interface is not the primary streaming/video test surface.

Exposure

Enable Swagger and its OpenAPI document in development by default. Any shared test environment requires restricted access. Keep both disabled in public production unless deliberately enabled behind access controls. This follows Microsoft guidance for local ad-hoc use [9].

09 / Google authentication and authorization

Identity and access have distinct responsibilities

Use Google sign-in through OpenID Connect on OAuth 2.0. Google verifies the user identity; ASP.NET Core policies enforce which RepoAI workspaces, repositories, videos and chats that user may access. Google OAuth scopes govern Google resources, not RepoAI project roles [10,11].

High-level login flow

Browser → RepoAI login → Google consent/sign-in → backend callback → RepoAI session. Register a Google web client with exact HTTPS callback URLs for each environment. Use ASP.NET Core OpenID Connect middleware with authorization code flow and PKCE, requesting openid, profile and email. Keep the client secret on the server.

Middleware validates issuer, audience, signature, expiry, state and nonce. Map the stable Google issuer + subject to a local RepoAI user; do not use email as the immutable account key or auto-link accounts solely by email. Issue a Secure, HttpOnly application session cookie and use anti-forgery protection for state-changing requests.

Authorization model

RepoAI role	Proposed permission boundary
Owner	Manage workspace membership, configuration and project lifecycle.
Editor	Import repositories, generate videos and use chat for assigned workspace projects.
Viewer	Read authorized projects/videos and ask questions; no imports or destructive changes.

Persist membership and role assignments locally. A successful Google login does not grant access to another user's workspace. Enforce policies on every command, evidence read, chat tool and artifact download; recheck active sessions after role revocation.

Session and provider boundaries

Google access tokens are for Google APIs and are not accepted directly as RepoAI authorization tokens. Avoid retaining them when only sign-in is needed. Local logout clears the RepoAI session; it does not log the user out of Google globally. Google sign-in also does not grant access to private GitHub/GitLab repositories; those need a separate repository credential connection.

10 / LTX video-generation integration

Model selection and runtime

LTX is the selected video model family. The official LTX-2 repository provides a Python inference stack for audio-video generation [12]. Pin the chosen checkpoint, runtime, quantization and model license after a capability benchmark. RepoAI stays a .NET application, with LTX explicitly isolated as an external inference dependency.

Integration boundary

A C# video adapter submits versioned scene requests to a private inference bridge, polls progress, requests cancellation and receives immutable artifact references. The HTTP job bridge is an integration component to implement or select; the upstream model package is not assumed to expose RepoAI's job API. Only trusted templates control runtime parameters.

Record the prompt, checkpoint hash, seed, input image hashes, dimensions, frame count and runtime version with each clip. Seeds aid reproducibility but do not guarantee identical output across runtime/hardware changes. Cache completed clips only when all relevant inputs match.

Accuracy, narration and output quality

Generate visual clips from the approved storyboard. Composite exact code, service labels and evidence-linked diagrams afterwards through deterministic templates. Validate duration, dimensions and decodability, then align narration and captions using actual media timing. Use a controlled narration track for exact technical wording; optional generated sound must not introduce unverified explanations.

LTX generates raster video, not an editable 3D scene graph. Interactive architecture views still use structured graph data and browser 3D templates. Assess generated scenes for motion consistency and visual relevance; preview and regenerate individual clips rather than automatically recreating an entire video.

GPU scheduling and distribution

LTX runs in its own GPU queue with bounded attempts, cancellation and scene-level checkpoints. Persist progress outside GPU memory. Benchmark warm/cold start, memory usage and per-clip latency at the target quality profile. Prefer dedicated capacity so video diffusion does not stall conversational inference.

Application source licensing and model licensing are separate. Check the exact LTX checkpoint's applicable terms before redistribution; LTX-2 publishes a community model license [13]. Mount weights separately from RepoAI images and record download provenance. Local inference avoids sending repository-derived prompts to a hosted provider.

11 / Technology references

Baseline sources were checked on 22 September 2026; Swagger, Google identity and LTX references were checked on 25 September 2026. Architecture choices and performance targets are proposals, not benchmark results.

[1] Microsoft .NET support policy
https://dotnet.microsoft.com/en-us/platform/support/policy

[2] Microsoft.Extensions.AI: IChatClient
https://learn.microsoft.com/en-us/dotnet/ai/ichatclient

[3] Roslyn semantic analysis
https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/work-with-semantics

[4] Playwright .NET browser capture
https://playwright.dev/dotnet/docs/screenshots

[5] Docker Compose GPU support
https://docs.docker.com/compose/how-tos/gpu-support/

[6] Native llama.cpp server
https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md

[7] Npgsql Entity Framework Core provider
https://www.npgsql.org/efcore/release-notes/10.0.html

[8] PostgreSQL full-text indexes
https://www.postgresql.org/docs/17/textsearch-indexes.html

[9] Microsoft: Swagger UI with ASP.NET Core OpenAPI (.NET 10)
https://learn.microsoft.com/en-us/aspnet/core/fundamentals/openapi/using-openapi-documents?view=aspnetcore-10.0

[10] Microsoft: Google external login (.NET 10)
https://learn.microsoft.com/en-us/aspnet/core/security/authentication/social/google-logins?view=aspnetcore-10.0

[11] Google: OpenID Connect
https://developers.google.com/identity/openid-connect/openid-connect

[12] LTX-2 official inference repository
https://github.com/Lightricks/LTX-2

[13] LTX-2 community model license
https://github.com/Lightricks/LTX-2/blob/main/LICENSE-2

Document boundary

The next LLD can define concrete API contracts, class responsibilities, database schema, event formats and job state transitions. This HLD intentionally stays at component and system-design level.